Lab Manual

Intelligent Mobile Robots (0135343)

2026

Faculty of Science & IT

Prepared By: Dr. Akram Fatayer

Revised By:

Al-Zaytoonah Private University of Jordan

Faculty of Science and Information Technology

Lab Manual

Course Name: Intelligent Mobile Robots

Course Number: 0135343

Department Name: Data Science and Artificial Intelligence

Prepared by: Dr. Akram Fatayer

Table of Contents

Number	Subject	Page
1	Lab Manual Overview	3
2	Required Tools and Resources	4
3	Rules and Policies	5
4	Learning Outcomes	5
5	Exercise 1: Hello, Robot	7
6	Exercise 2: Localization: Fusing Odometry and GPS	9
7	Exercise 3: Vision-Based Localization	11
8	Exercise 4: Mapping: An Occupancy Grid from LiDAR	13
9	Exercise 5: Path Planning: A* and RRT	14
10	Exercise 6: Path Tracking and Control	15

Lab Manual Overview

The labs build, step by step, the software of an autonomous warehouse robot in simulation. Each lab adds one part of the Sense-Plan-Act loop: moving and sensing, estimating where the robot is, seeing landmarks, mapping, planning a path and following it. The same parts are then joined in the course project. Labs run in Python with PyBullet in the shared zuj-robotics environment installed on every lab PC, and also in Google Colab; Labs 1 and 3 have an optional MATLAB track.

The six experiments in this manual are the graded laboratory experiments of the course. Each one is explained by the instructor, carried out by students in the lab session, and assessed by an assignment linked to that experiment (code outputs, plots and short written answers), worth 5 marks: 30 marks in total.

Course-specific learning goals of the lab:

- control a simulated mobile robot and read its sensors;
- estimate the robot's position by fusing noisy sensors with a Kalman filter, including landmarks seen by its camera;
- build an occupancy-grid map from LiDAR, plan paths with A* and RRT, and track them with PID and Pure Pursuit;
- judge every method by measured error (pose error, path length, cross-track error), not by how the animation looks.

Required Tools and Resources

Number	Name
1	**Lab PC environment**: The shared zuj-robotics Conda environment on every lab PC (Python 3.11, PyBullet, OpenCV, NumPy, Matplotlib); open it with the Robotics Lab desktop shortcut.
2	**Google Colab**: Every lab also runs in Colab for work outside class; no installation needed.
3	**Own laptop (optional)**: Install the same environment once from SETUP.md in the course repository, then run check_env.py.
4	**MATLAB track (optional)**: MATLAB R2025a or later with Simulink for Labs 1 and 3, through MATLAB Online or the lab installation.
5	**Course repository**: github.com/akramfatayer/zuj-robotics: lab scripts and notebooks, lecture slides and the project starter code.
6	**Course page and interactive modules**: akramfatayer.github.io/courses/robotics: browser-based modules for each block.
7	**Moodle**: Submission of every graded experiment and feedback on it.

Rules and Policies

❖ Before you come to the lab session

- You can bring your notebook so that you can keep your work with you
- No food or beverage is allowed in the lab
- Be sure to come on time to make sure you follow lab exercises with other students.
- Be sure to remember your student account username and password

❖ While being in the lab session

- Interact with the lab supervisor. Do not hesitate to ask him questions
- Maintain silence and discipline while being in the lab
- Show cooperation with your fellow students
- Log in to your computer using your student account
- Make sure you complete your lab exercises

❖ After finishing the lab session

- Make sure the place you are using is clean
- Put your chair or any material you have used back to its proper place
- Make sure to logout of your student account

❖ Attendance

- You are required to come on time, otherwise you will be considered absent
- Absence from lab sessions is recorded and will be used for absence warning system

❖ Grading

- Each of the six graded experiments (Exercises 1–6) is worth 5 marks and is assessed by an assignment linked to that experiment: 30 marks in total.
- Submit each graded experiment on Moodle by the start of the next lab session. Late work loses 10% per day for up to three days.
- Marks are recorded on the Lab Experiment Evaluation Form. You may be asked to run and explain your code.

Learning Outcomes

Exercise Number	Learning Outcome

IT LAB Manual (Intelligent Mobile Robots)

Exercise 1	Spawn a robot in PyBullet, read its sensors and drive a square; explain the Sense-Plan-Act loop (K1, S1).
Exercise 2	Implement a Kalman filter that fuses odometry and GPS, and measure its error against each sensor alone (K2, S2).
Exercise 3	Detect wall markers in the robot's camera images and use them as measurements to correct the Kalman filter (K2, S2).
Exercise 4	Build an occupancy-grid map of the warehouse from simulated LiDAR (K2, S1).
Exercise 5	Implement A* and RRT and compare them in a simulated warehouse (K3, S1).
Exercise 6	Implement PID and Pure Pursuit, tune them and compare their tracking error (K4, C2).

Exercise 1: Hello, Robot

Week 2 · Folder: labs/lab-01-hello-robot (also in Colab and MATLAB) · Graded by: assignment (code, video and short report), 5 marks

Exercise Objectives:

- Set up the simulation environment.
- Spawn a robot model (URDF) in PyBullet.
- Read the robot's position, orientation and camera image.
- Send velocity commands to drive a square, and analyse the path it really took.

Background:

Every autonomous robot runs a Sense-Plan-Act loop. A physics simulator lets us test that loop safely: the robot is described by a URDF model, its sensors return the pose and camera images, and its actuators take wheel-velocity commands. Driving a square open loop, without feedback, shows how small errors build up.

In class Experiments (for testing):

- Part 1: create the simulation and spawn the robot (spawn_robot.py).
- Part 2: read and print the sensor data (read_sensors.py).
- Part 3: drive the robot in a square (move_robot.py).

Experiment #1 (for grading):

Assignment (5 marks): all four completed scripts, a short screen recording of the robot driving the square, and a 1–2 page report with the trajectory plot from visualize_trajectory.py.

- Working spawn, sensing and motion scripts (2 marks).
- Trajectory plot and video (1.5 marks).
- Report: how far the robot ended from where it started, and why (1.5 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- The sensor readings, the trajectory plot and the final position error.

Discussion / Conclusion:

- Why does the square not close exactly?

- Which part of Sense-Plan-Act is missing from an open-loop square?

Exercise 2: Localization: Fusing Odometry and GPS

Week 5 · Folder: labs/lab-03-localization (also in Colab and MATLAB) · Graded by: assignment (plots and answers), 5 marks

Exercise Objectives:

- Apply the Kalman filter's predict and update steps in 1D and in 2D with matrices.
- Use odometry as the control input and GPS as the measurement.
- Measure the filter's error against odometry alone, GPS alone and a gated reset.

Background:

Odometry is fast and smooth but drifts; GPS never drifts but is noisy and slow. The Kalman filter keeps a Gaussian belief about the robot's position: it predicts with odometry, which increases its uncertainty, and updates with each GPS fix, weighting it by the Kalman gain. The process noise Q and measurement noise R decide whom the filter trusts.

In class Experiments (for testing):

- Activity 1: the 1D filter on a robot driving straight (activity1_straight_1d.py).
- Activity 2: the 2D filter, matrix by matrix, then on R2D2 driving an L-shaped path, with GPS and with a wall sensor (activity2_pybullet_2d_v3.py).
- Activity 3: odometry and GPS fused in a 60 m aisle (activity3_pybullet_fusion.py).

Experiment #2 (for grading):

Assignment (5 marks): the three plots and the answers to the activity questions in the lab README.

- Activity 1 plot and answers (1 mark).
- Activity 2: the Kalman gain checked by hand, both sensor runs and answers (2 marks).
- Activity 3: results with changed GATE, Q and GPS_STD, and answers (2 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- The estimate and ±2σ band in 1D, the 2D uncertainty ellipses, and the mean error of the four estimators in the aisle.

Discussion / Conclusion:

- Why is the Kalman filter's error about 1 m when each sensor alone gives about 3 m?

- What happens when Q is far too small or far too large?

Exercise 3: Vision-Based Localization

Week 6 · Folder: labs/lab-03b-vision-localization (to be released) · Graded by: assignment (plots and answers), 5 marks

Exercise Objectives:

- Detect coloured markers on the walls of the simulated arena in the robot's camera images.
- Turn a detected marker into a measurement of the robot's position, using the camera model and the marker's known location.
- Use these measurements to correct the Kalman filter, and compare the result with GPS correction.

Background:

A robot indoors often has no GPS, but it can see landmarks. Colour thresholding in HSV, contours and centroids (Lecture 2) find a marker in the image; the pinhole camera model turns its pixel column into a bearing and its apparent size into a distance. Because the marker's position on the map is known, each sighting gives a position measurement that the Kalman filter's update step can use exactly like a GPS fix, with its own noise R.

In class Experiments (for testing):

- Activity 1: capture camera frames as the robot drives, threshold the marker colours and draw each marker's contour and centroid.
- Activity 2: compute the bearing and range to each detected marker and convert them to a position estimate; check it against the true pose.
- Activity 3: run the Kalman filter with odometry in the predict step and marker measurements in the update step; compare it with odometry alone and with GPS updates.

Experiment #3 (for grading):

Assignment (5 marks): the detection images, the position plot and the answers.

- Marker detection working on the camera frames (1.5 marks).
- Bearing and range to position, with the error against the true pose (1.5 marks).
- Kalman filter corrected by markers, compared with odometry alone and GPS (2 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- Camera frames with the detected markers, the measurement error, and the mean pose error of each estimator.

Discussion / Conclusion:

- What happens to the estimate when no marker is in view for a while?
- Which gives the better correction here, markers or GPS, and why?

Exercise 4: Mapping: An Occupancy Grid from LiDAR

Week 7 · Folder: labs/lab-04-mapping (also in Colab) · Graded by: assignment (maps and answers), 5 marks

Exercise Objectives:

- Read and visualise simulated LiDAR scans.
- Convert range readings from the robot's frame to world and grid coordinates.
- Build an occupancy grid of the warehouse as the robot drives.

Background:

A LiDAR measures the distance to the nearest obstacle along many directions. Each reading, converted from polar to world coordinates, marks the cell where the beam stopped as occupied. An occupancy grid divides the floor into cells and records which are free, occupied or unknown; it is the map the planner of Lab 5 will search.

In class Experiments (for testing):

- Activity 1: LiDAR visualisation and coordinate transformation (activity1_lidar_viz.py).
- Activity 2: a basic occupancy grid with world-to-grid conversion (activity2_basic_grid.py).
- Activity 3: real-time mapping of the warehouse in PyBullet (activity3_warehouse_mapping.py).

Experiment #4 (for grading):

Assignment (5 marks): the three output images and the answers.

- Activity 1 point cloud and coordinate check (1 mark).
- Activity 2 grid with correct world-to-grid conversion (1.5 marks).
- Activity 3 warehouse map and the effect of grid resolution (2.5 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- The LiDAR plot, the basic grid and the warehouse map at two resolutions.

Discussion / Conclusion:

- What does a finer grid cost, and what does it buy?
- How would a wrong robot pose show up in the map?

Exercise 5: Path Planning: A* and RRT

Week 11 · Folder: labs/lab-05-path-planning (also in Colab) · Graded by: assignment (code, plots and answers), 5 marks

Exercise Objectives:

- Implement A* from scratch and explain g(n), h(n), f(n) and admissibility.
- Implement RRT and compare it with A*.
- Plan a path through the warehouse and show it in PyBullet.

Background:

A* searches a grid for the shortest path, guided by a heuristic that never overestimates the remaining distance; with an admissible heuristic it is optimal. Grids grow exponentially with dimension, so for robot arms sampling-based planners such as RRT grow a random tree instead and find a feasible, not optimal, path.

In class Experiments (for testing):

- Activity 1: A* on a grid, with Manhattan and Euclidean heuristics (activity1_astar.py).
- Activity 2: RRT on the same map (activity2_rrt.py).
- Activity 3: A* in the 3D warehouse simulation (activity3_warehouse.py).

Experiment #5 (for grading):

Assignment (5 marks): the completed scripts, the path plots and the answers.

- A* correct and optimal on the test maps (2 marks).
- RRT working, with a comparison of path length and run time (1.5 marks).
- Warehouse path and answers (1.5 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- Path length, nodes expanded and run time for A* and RRT on the same map.

Discussion / Conclusion:

- Why is A* optimal only with an admissible heuristic?
- When would you choose RRT over A*?

Exercise 6: Path Tracking and Control

Week 13 · Folder: labs/lab-06-tracking-control (also in Colab) · Graded by: assignment (code, plots and answers), 5 marks

Exercise Objectives:

- Define cross-track error and heading error.
- Implement a PID controller on cross-track error and tune its gains.
- Implement Pure Pursuit, compare it with PID and drive a full warehouse route planned with A*.

Background:

A planner gives waypoints; a controller turns them into wheel commands. PID reacts to the cross-track error with proportional, integral and derivative terms. Pure Pursuit steers towards a point a look-ahead distance ahead on the path; a short look-ahead tracks tightly but oscillates, a long one is smooth but cuts corners.

In class Experiments (for testing):

- Activity 1: PID on cross-track error, with gain tuning (activity1_pid.py).
- Activity 2: Pure Pursuit with different look-ahead distances (activity2_pure_pursuit.py).
- Activity 3: Pure Pursuit following an A* route in the warehouse (activity3_warehouse.py); check with check_my_lab.py.

Experiment #6 (for grading):

Assignment (5 marks): the completed scripts, the output of check_my_lab.py, the plots and the answers.

- PID tuned to a well-damped response (1.5 marks).
- Pure Pursuit with the look-ahead trade-off shown (1.5 marks).
- Warehouse route tracked to the goal, with its mean cross-track error (2 marks).

Zip the listed files and submit them on Moodle by the start of the next lab session.

Results / Observations:

- Cross-track error over time for each controller and setting, and the mean error on the warehouse route.

Discussion / Conclusion:

- What does each PID term fix, and what does it cost?
- How does the look-ahead distance trade accuracy for smoothness?